

PROJECT AND TEAM INFORMATION

Project Title

(Try to choose a catchy title. Max 20 words).

MazeQuest

Student / Team Information

<i>Team Name:</i> <i>Team #</i>	MazeMind
Team member 1 (Team Lead) (Last Name, name: student ID: email, picture):	Panwar, Kartikey – 2510010186 panwarkartikey3@gmail.com 
Team member 2 (Last Name, name: student ID: email, picture):	Sharma, Aadi – 2510380150 aadi10.Doon@gmail.com 
Team member 3 (Last Name, name: student ID: email, picture):	Choudhary, Paridhi – 2510010449 1paridhichoudhary@gmail.com 

PROPOSAL DESCRIPTION (10 pts)

Motivation (1 pt)

(Describe the problem you want to solve and why it is important. Max 300 words).

C programs often end at console-based linked lists, queues, stacks, or graph traversal, while C++ projects may focus on classes and screens without showing how the data is actually represented. Our project tries to connect both sides through a game. In our project, Graph-Based Survival Maze, the maze is stored as a graph: every walkable position is a vertex, and every possible movement to a neighbouring position is an edge. The survival part makes the graph useful during play, because the player starts with limited stamina and cannot simply walk straight to the exit. They have to explore, collect fruits, and choose routes carefully. This makes BFS, DFS, pathfinding, and graph representation feel connected to a working application instead of only being theory written on paper.

State of the Art / Current solution (1 pt)

(Describe how the problem is solved today (if it is). Max 200 words).

Most beginner maze programs are made using a 2D array, a fixed map, and a simple recursive backtracking method. These are good for learning the basic idea of a maze, but they do not always make the graph structure clear. Many examples also solve the maze automatically, so the player does not really face decisions. Commercial survival or puzzle games do have health, energy, scoring, and levels, but the algorithms behind movement and pathfinding are hidden from the user and are not meant as DSA demonstrations. Our project is different because the maze is not only drawn on screen; it is built around adjacency lists, queues, stacks, BFS, DFS, and linked lists, while still having menus, login, score storage, difficulty choices, and an interactive Raylib interface.

Project Goals and Milestones (2 pts)

(Describe the project general goals. Include initial milestones as well any other milestones. Max 300 words).

Our final goal is to make a maze solver that uses the concepts of both OOPs and Data Structures together as a single project. Data structures including, graphs, adjacency lists, queue, stack and techniques like BFS, DFS, PathFinding and our maze generation would be made entirely in C. C++ will be used for classes, states of game and for the interaction between them.

Our first milestone would be to finish the graph structure and try the solver on small mazes, further to ensure the C logic is connected to the classes in C++. Then we will add raylib rendering, raygui controls and the login/signup in the database along with the stamina and recovery.

Project Approach (3 pts)

(Describe how you plan to articulate and design a solution. Including platforms and technologies that you will use. Max 300 words).

Our maze logic would be in C, we will represent the maze in the form of a grid, which would then be converted to a graph for the user to traverse ,(the walkable cells would be the vertices).

Each individual vertex would be connected via an adjacency list. We will use BFS to find the shortest path and confirm whether the maze is solvable. DFS would also be implemented for traversal and maze generation experiments.

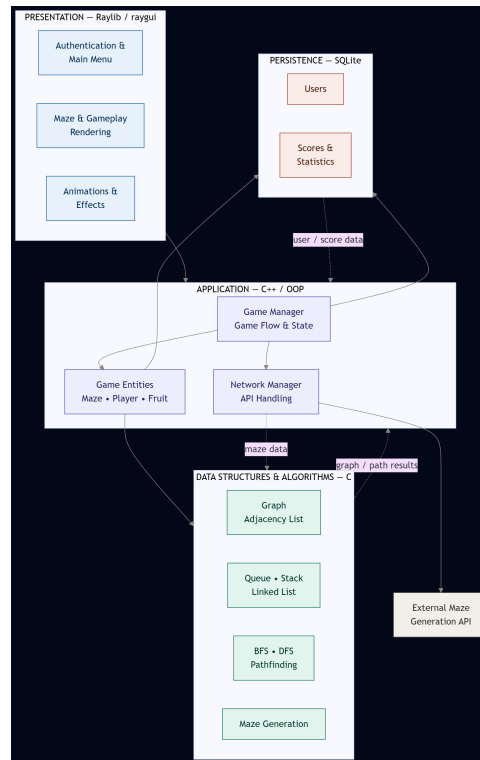
We will use C++ to handle the class layer notable: a class Game will control the current screen and main loop. Similarly

classes like Maze, Player, Fruit and User classes will store related data and behaviour, as per required. For the maze drawing and fruits, player, effects we will be using Raylib.

Raylib will be used for drawing the maze, player, fruits, menus, animations, and effects. raygui will help with buttons, text boxes, dropdowns, and sliders. SQLite will store user accounts, scores, and statistics. For API mode, the Network Manager will request maze data and convert it into the same graph format used by the manual generator.

System Architecture (High Level Diagram)(2 pts)

(Provide an overview of the system, identifying its main components and interfaces in the form of a diagram using a tool of your choice).



Our system would be based on a layered OOPs based design, each part would have a fixed clear job instead of all the logic being mixed in a single file. For our GUI we will use Raylib and Raygui, for stuff like login, menu, gameplay, drawing, animations, effects, user input etc. The Cpp layer would control the flow of the game, acting as a game manager based on classes like Maze, Fruits, Player etc.

Our C layer would contain our representation of graphs, other data structures needed for the project like: Adjacency lists, Queues, Stacks, Linked lists, BFS/DFS/Pathfinding/Maze generation logic, SQLite for persistent database (for entries like user records, score, leaderboard, login). If the API mode is selected then the maze data is received from external sources and converted in our original graph representation used by our underlying logic.

Project Outcome / Deliverables (1 pts)

(Describe what are the outcomes / deliverables of the project. Max 200 words).

The final outcome of our project is a working desktop game. Users sign up or log in, choose a maze mode, pick a difficulty, and start playing. From there it's a stamina challenge — move through the maze, collect fruit and other resources to keep stamina up, and try not to run out before reaching the goal. Raylib handles the visuals: the maze itself, the player, resources on the ground, a stamina bar, and the win/loss screen at the end.

Behind that interface, the maze isn't just a picture — it's a graph, built and traversed using data structures and algorithms we implemented ourselves in C.

Deliverables:

Playable executable

Source code

SQLite database design

Maze generation support (both API-driven and manual)

Short build instructions

A short writeup on where BFS, DFS, stacks, queues, linked lists, and OOP classes were used

Assumptions

(Describe the assumptions (if any) you are making to solve the problem. Max 100 words)

We're assuming the target machine can run Raylib with basic graphics support and can create a local SQLite database file — nothing exotic there. We're also assuming the maze-generation API is available during testing and returns data we can actually convert into our graph structure. If it's not available, though, we're not stuck: the game still works fine through manually generated mazes.

References

(Provide a list of resources or references you utilised for the completion of this deliverable. You may provide links).

Raylib official documentation

raygui repository and documentation

SQLite C/C++ interface documentation

GeeksforGeeks notes on graph data structures and algorithms

cppreference C++ language reference